1. Resumen Ejecutivo del Tablero Scrum

Hito / Punto	Historia de Usuario (Jira)	Estimación	Estado	Entregable Clave
Punto 4A	US-DB-04: Contenerización Motor PostgreSQL y pgAdmin	5 SP	**DONE**	PR #1 Merged en develop (Compose + 16 tablas + pgAdmin)
Punto 4B	US-DB-05: Dataset Masivo de 6M y Carga Distribuida	5 SP	**DONE**	PR #6 Abierto hacia develop (seed-6m.mjs distribuido en 20k usuarios)
Punto 4C	US-DB-06: Aislamiento Loopback y Scripting de Métricas	3 SP	**DONE**	Auditoría de cardinalidad y tamaño físico en disco (996 MB)

Velocidad del Integrante: 13 Story Points completados satisfactoriamente con entrega integral de infraestructura y dataset del Punto 4.

2. Detalle Exhaustivo por Historia de Usuario

PUNTO 4A: Contenerización Motor PostgreSQL y pgAdmin (US-DB-04)

> *"Como docente evaluador y equipo de desarrollo, quiero disponer del motor de base de datos PostgreSQL 17 y un cliente visual pgAdmin 4 aislados en contenedores Docker y orquestados con compose.yaml, para garantizar un entorno reproducible, persistente y seguro."*

Criterios de Aceptación Cumplidos (DoD):

- Despliegue de imagen oficial ligera postgres:17-alpine (gymbro-postgres-primary) con healthcheck pg_isready.
- Despliegue de cliente administrativo dpage/pgadmin4:latest (gymbro-pgadmin) condicionado al arranque saludable del motor (service_healthy).
- Persistencia de datos física garantizada mediante volúmenes nombrados smartgymbro_tp_postgres_data y smartgymbro_tp_pgadmin_data.
- Red interna de Docker bridge (gymbro_network) permitiendo resolución DNS interna (postgres-primary:5432).
- Seguridad y aislamiento perimetral en Loopback (127.0.0.1:55433 y 127.0.0.1:5051), impidiendo exposición indebida a redes externas o Wi-Fi.
- Aprovisionamiento automático de esquema gymbro con 16 tablas relacionales y catálogo de 190 ejercicios oficiales mediante scripts de inicialización (01-bootstrap.sql y 08-nombres-espanol.sql).

Bloqueos Resueltos en la Daily:

- **Sintaxis restrictiva en Compose original:** Se eliminó la traba ${POSTGRES_DB:?...} desacoplando limpiamente las credenciales hacia postgres/.env.
- **Error DATABASE_URL no configurada:** Se configuró backend/.env con la cadena de conexión local hacia el puerto 55433 en modo LOCAL.
- **Validación RFC en pgAdmin:** Se corrigió el correo por defecto a admin@smartgymbro.com evitando rechazos de nombres especiales de dominio.

Evidencia: PR #1 aprobado y fusionado en develop (Commits fd9437e y 8d3165e, Merge 5be5421). Verificación local con npm run postgres:verificar (16 tablas y 190 ejercicios).

PUNTO 4B: Inyección Masiva Distribuida de 6.000.000 de Registros (US-DB-05)

> *"Como equipo de desarrollo y docentes evaluadores, quiero inyectar 6.000.000 de registros sintéticos en la tabla de movimiento del sistema distribuidos de forma realista sobre una población de usuarios, para simular estrés de producción y permitir la optimización con EXPLAIN ANALYZE en el Punto 5."*

Criterios de Aceptación Cumplidos (DoD):

- Selección fundamentada de la Tabla de Movimiento: gymbro.historial_peso_corporal (serie temporal de pesajes transaccionales).
- Generación previa de población en Tabla Maestra: 20.000 usuarios en gymbro.usuarios con correos únicos, alturas, objetivos y claves foráneas válidas.
- Inyección masiva de 6.000.000 de filas mediante producto cartesiano de alto rendimiento (CROSS JOIN generate_series(1, 300)) en memoria del motor PostgreSQL (C nativo).
- Distribución simétrica y realista: Exactamente 300 pesajes por usuario a lo largo de 2 años (mínimo: 300, máximo: 300, promedio: 300.00).
- Validación estricta de Check Constraint: Valores de peso entre 55.00 kg y 100.00 kg (cumple CHECK peso_kg > 0 AND peso_kg <= 500).
- Métricas físicas medidas: 438 MB de datos de tabla + 558 MB de índices B-Tree = 996 MB total tabla (~1.0 GB en disco).
- Cardinalidad óptima para Punto 5: 20.000 usuarios distintos posibilitando pruebas de B-Tree Index Scan vs Sequential Scan.

Bloqueos Resueltos en la Daily:

- **Hoisting en ECMAScript Modules (.mjs):** Los imports estáticos se procesaban antes de dotenv.config(); se resolvió mediante importación dinámica asíncrona (await import('../backend/src/db.js')).
- **Defecto conceptual de usuario único:** La carga inicial concentraba 6M de pesajes en 1 solo usuario (cardinalidad 1). Se vació la tabla con TRUNCATE TABLE atómico (24 kB) y se implementó la arquitectura distribuida en 20.000 usuarios.
- **Depuración de caracteres y emojis:** Se removieron caracteres unicode del script para garantizar compatibilidad con consolas de producción y UTF-8.

Evidencia: Script postgres/seed-6m.mjs validado. Ejecución de carga masiva en ~60s con 6.000.000 registros y 20.001 usuarios. PR #6 abierto hacia develop (Commit fix(db)).

3. Guía de Carga para Gonzalo en Jira y Nextcloud (Punto 2)

- **Tablero Kanban / Scrum:** Trasladar US-DB-04 a la columna DONE, y US-DB-05 a IN REVIEW / DONE tras aprobación del PR.
- **Velocidad y Burndown:** Asignar los 13 Story Points completados para ilustrar el progreso real del Sprint en la capa de datos.
- **Retrospectiva del Sprint:** Destacar como fortaleza la técnica de producto cartesiano con generate_series (más de 100.000 registros/segundo en memoria de base de datos) y el aislamiento estricto de seguridad en Loopback.

———

4. Guía Rápida para el Equipo:

A. ¿Por qué se hizo así y cuál es la tabla de movimiento?

En una app de gimnasio como Gymbro, los usuarios se crean una sola vez (tabla maestra usuarios), pero a medida que van a entrenar o hacen dieta, se pesan periódicamente en la balanza. Ese pesaje no se sobreescribe: cada pesaje es un evento histórico nuevo en la tabla historial_peso_corporal (tabla de movimiento). Para que la base de datos sea 100% realista y nos sirva en el Punto 5 para optimizar con EXPLAIN ANALYZE, el script no mete 6 millones de pesajes a un solo usuario inventado, sino que primero crea 20.000 usuarios atletas y después le asigna exactamente 300 pesajes a cada uno a lo largo de 2 años. Así los índices B-Tree funcionan de verdad.

B. Comandos exactos para replicar todo en 2 minutos

Paso 1: Levantar los contenedores de Docker

Con Docker Desktop abierto, corré desde la raíz del repo:

npm run postgres:up
(Esto levanta PostgreSQL 17 en el puerto 55433 y pgAdmin en el 5051 sin colisionar con otros puertos locales).

Paso 2: Comprobar que se crearon las 16 tablas y los 190 ejercicios

Ejecutá el script de validación del TP:
npm run postgres:verificar
Tiene que devolver en verde: {"ok":true,"tablas":16,"ejerciciosOficiales":190}.

Paso 3: Generar los 6 millones de registros distribuidos

Ejecutá el script generador de datos:
node postgres/seed-6m.mjs
Tarda ~60 segundos. Va a crear los 20.000 usuarios y los 6.000.000 de pesajes en memoria C de Postgres y te va a imprimir las métricas de disco (~1 GB total).

Paso 5: Cómo entrar al pgAdmin a ver las tablas

1. Abrí en el navegador: http://localhost:5051

Credenciales omitidas. Consultar configuracion privada.

3. Para registrar el servidor: Host: postgres-primary (OJO: no poner localhost porque corre adentro de Docker),

Credenciales omitidas. Consultar configuracion privada.

4. Vas a Servers -> smartgymbro_tp -> Schemas -> gymbro -> Tables y ves las 16 tablas con los 6 millones de filas.

```
PS C:\Users\facun\Downloads\gymbro\tp-Final-Gymbro> node postgres/seed-6m.mjs
================================================================
SMARTGYMBRO - INYECCION MASIVA DISTRIBUIDA (6.000.000 FILAS)
================================================================

[OK] Conexion establecida con PostgreSQL 17 en Docker.

[1/2] Verificando poblacion de usuarios en la tabla maestra...
Generando 20.000 usuarios sinteticos en gymbro.usuarios...
[OK] 20.000 usuarios creados exitosamente en 1.30 segundos.

[2/2] Verificando tabla de movimiento (historial_peso_corporal)...
Registros actuales: 0
Iniciando inyeccion masiva de 6.000.000 de pesajes...
[INFO] Distribucion: 20.000 usuarios x 300 pesajes historicos cada uno.
Por favor espera unos 45-60 segundos mientras PostgreSQL procesa en memoria...

[OK] Inyeccion masiva completada con exito en 579.14 segundos!
Rendimiento: 10.360 registros por segundo.

================================================================
METRICAS TECNICAS
================================================================
- Total usuarios maestros     : 20.001 usuarios
- Total registros movimiento  : 6.000.000 pesajes
- Promedio por usuario        : 300 pesajes (~1 cada 2 dias en 2 anos)
- Tamano de datos de tabla    : 438 MB
- Tamano de indices B-Tree    : 558 MB
- Tamano total de la tabla    : 996 MB
- Tamano total base de datos  : 1011 MB
================================================================
```